

Reporte Técnico y Manual de Usuario: StudentWallet

Institución: Universidad Autónoma de Chihuahua (UACH)

Facultad: Facultad de Ingeniería

Programa Educativo: Ingeniería en Ciencias de la Computación

Asignatura: Cómputo en la Nube (Cloud Computing)

Autor: Idaly Guadalupe Morales Robredo

Fecha: Mayo de 2026

1. Descripción General del Proyecto

StudentWallet es una plataforma web integral diseñada para la gestión, monitoreo y análisis de las finanzas de estudiantes universitarios. El sistema permite registrar ingresos y gastos, definir presupuestos mensuales con alertas de consumo, realizar un seguimiento interactivo de metas de ahorro y obtener un análisis financiero avanzado mediante una arquitectura híbrida de Cómputo en la Nube. El objetivo primordial es dotar al estudiante de una herramienta accesible pero con estándares profesionales de seguridad y un procesamiento de datos robusto respaldado por servicios serverless.

1.1. Arquitectura de Software y Tecnologías

La aplicación adopta una arquitectura desacoplada para separar las responsabilidades de presentación, almacenamiento y cómputo avanzado:

- **Capa de Presentación y Lógica Web:** Desarrollada en PHP 8.2 con un diseño responsivo basado en Tailwind CSS y gráficos dinámicos interactivos mediante Chart.js.
- **Capa de Datos:** Implementada en un motor relacional MySQL, estructurado bajo estrictas restricciones de integridad y normalización de bases de datos.
- **Capa de Cómputo Serverless:** Una función en AWS Lambda escrita en Python 3.9 encargada del procesamiento estadístico pesado y la detección de anomalías financieras.
- **Capa de Almacenamiento de Archivos:** Integración con un bucket de Amazon S3 para el resguardo seguro de documentos académicos y ensayos, distribuidos mediante URLs firmadas temporalmente.

2. Modelo de Base de Datos (Esquema Relacional)

La persistencia de los datos del sistema se encuentra completamente normalizada e indexada en una base de datos denominada studentwallet, codificada en un esquema global utf8mb4_unicode_ci para garantizar compatibilidad total con caracteres especiales.

2.1. Estructura de Tablas

El modelo relacional consta de las siguientes entidades estructuradas mediante el archivo schema.sql:

Tabla	Descripción Técnico-Funcional	Restricciones e Índices
usuarios	Almacena las credenciales y perfiles de los usuarios de la plataforma.	PRIMARY KEY (id), UNIQUE KEY (username).
intentos_login	Registra los accesos fallidos y exitosos por dirección IP para auditoría y seguridad.	Índice compuesto idx_intentos_ip_fecha (ip, intentado_en).
categorias	Clasifica los movimientos financieros. Soporta categorías del sistema (globales) y personalizadas por usuario.	FOREIGN KEY (usuario_id) REFERENCES usuarios(id), UNIQUE KEY uk_categoria_usuario (usuario_id, nombre).
ingresos	Almacena los flujos de efectivo entrantes (becas, mesadas, trabajos libres).	CHECK (monto > 0), Índice idx_ingresos_usuario_fecha (usuario_id, fecha).
gastos_recurrentes	Gestiona los compromisos mensuales fijos y suscripciones (renta, plataformas de streaming).	CHECK (dia_del_mes BETWEEN 1 AND 31), Índice idx_recurrentes_activos (activo, fecha_inicio).
gastos	Entidad central que registra las salidas de dinero diarias del estudiante.	Índices idx_gastos_usuario_fecha e idx_gastos_categoria. Restricción chk_division para gastos compartidos.
presupuestos	Establece topes de gasto máximos por categoría y periodo temporal.	UNIQUE KEY uk_presupuesto (usuario_id, categoria_id, mes, anio).
metas_ahorro	Permite trazar metas de mediano y largo plazo vinculadas a un balance actual acumulado.	CHECK (monto_actual >= 0), FOREIGN KEY (usuario_id) REFERENCES usuarios(id).

2.2. Integridad Referencial y Restricciones Avanzadas

La base de datos delega de manera estricta las reglas de integridad al motor de

almacenamiento. Las relaciones críticas con la tabla de usuarios cuentan con la directiva ON DELETE CASCADE, garantizando que si una cuenta es eliminada, toda su información financiera relacionada se borre de manera limpia. Por el contrario, las categorías base implementan ON DELETE RESTRICT para impedir la eliminación accidental de clasificaciones esenciales utilizadas por múltiples transacciones activos. La restricción chk_division asegura mediante lógica booleana matemática que el monto individual registrado sea consistente con la división exacta del costo total entre el número de personas involucradas en consumos grupales.

3. Arquitectura Cloud e Infraestructura

El despliegue de StudentWallet se fundamenta en los pilares de alta disponibilidad, portabilidad e infraestructura como código mediante contenedores y servicios en la nube de Amazon Web Services (AWS).

3.1. Contenerización con Docker y Docker Compose

La aplicación web y el motor de base de datos se encuentran completamente aislados e interconectados mediante Docker:

- **Dockerfile:** Utiliza una imagen oficial de php:8.2-apache. Instala extensiones nativas necesarias como pdo_mysql, activa el módulo de reescritura de URLs de Apache (mod_rewrite), inyecta los archivos de configuración personalizados e incorpora la herramienta Composer para la instalación automatizada del SDK oficial de AWS (aws/aws-sdk-php).
- **docker-compose.yml:** Coordina el entorno definiendo dos servicios principales: app (servidor web expuesto en el puerto 80) y db (instancia de MySQL 8.0 protegida en una red privada interna). Administra volúmenes persistentes para los datos de la base de datos y la carpeta de uploads de archivos, inyectando de forma segura variables de entorno críticas como las credenciales de AWS y el nombre de la función Lambda asociada.

3.2. Configuración Avanzada del Servidor Web (Apache y PHP)

El entorno de producción cuenta con un endurecimiento robusto implementado en los archivos de configuración distribuidos en las carpetas correspondientes:

- **studentwallet.conf (Apache):** Configura el VirtualHost web apuntando el DocumentRoot única y exclusivamente a la carpeta public/. Por motivos de seguridad por diseño, implementa un bloqueo redundante absoluto mediante directivas Require all denied a todas las rutas internas sensibles del código que no deben tener acceso público web, tales como config/, vendor/, db/, lambda/ e infra/.
- **custom.ini (PHP):** Modifica las directivas de ejecución deshabilitando el envío de firmas de versión (expose_php = Off) para evitar ataques de reconocimiento. Fuerza a que las cookies de sesión se transmitan con las banderas session.cookie_httponly = 1 (previniendo robos vía scripts maliciosos), session.use_strict_mode = 1 y un alcance controlado session.cookie_samesite = "Lax".

3.3. Despliegue en AWS EC2 y Repositorio de Archivos en AWS S3

El entorno de producción se ejecuta sobre una instancia virtual de AWS EC2 con sistema operativo Ubuntu Linux, provisionada de forma ágil mediante el script automatizado `get-docker.sh`. El almacenamiento de documentos de la materia se delega a AWS S3 a través del script `descargas.php`. Cuando el sistema detecta la presencia de un bucket remoto mediante variables de entorno, deja de servir archivos localmente y se conecta al API de S3 generando de manera dinámica **URLs firmadas (Presigned URLs)**. Estas direcciones otorgan un token criptográfico seguro de acceso directo de lectura exclusivo para el objeto, parametrizado con una validez estricta de 15 minutos, mitigando la exposición de archivos a usuarios no autorizados.

3.4. Procesamiento Analítico Serverless con AWS Lambda

Una de las características innovadoras del proyecto es la delegación de operaciones matemáticas pesadas y analítica avanzada a un entorno Serverless distribuido:

- **Lógica en Python (`lambda_function.py`):** Diseñada para ejecutarse de forma aislada dentro de AWS Lambda utilizando la librería matemática nativa y `python-dateutil`. Recibe un JSON estructurado (payload) con los ingresos, gastos actuales, gastos del mes anterior, presupuestos activos y metas del usuario. Ejecuta un modelado completo que calcula proyecciones matemáticas al cierre del periodo e identifica anomalías basadas en desviación estándar.
- **Consumo desde PHP (`analisis_lambda.php`):** Implementa un puente híbrido. Mediante el uso del cliente `LambdaClient` del SDK de AWS, intercepta las llamadas en el archivo `analisis.php`. Si la variable de entorno `LAMBDA_FUNCTION_NAME` está configurada en la infraestructura, empaqueta todos los registros financieros recopilados del usuario y realiza una invocación síncrona mediante un canal cifrado de comunicación hacia la infraestructura de AWS en la región seleccionada, desempaquetando la respuesta JSON para renderizarla directamente en la interfaz de usuario. En caso de no estar configurada, el sistema cuenta con un fallback local automático (`analisis_local.php`) para entornos de desarrollo.

4. Seguridad y Defensas Implementadas (Security by Design)

El sistema fue construido bajo directrices estrictas de desarrollo seguro, mitigando de manera nativa las vulnerabilidades más comunes descritas en el estándar OWASP:

- **Protección contra Cross-Site Request Forgery (CSRF):** Cada sesión de usuario genera un token criptográfico seguro pseudoaleatorio e impredecible de 32 bytes (`bin2hex(random_bytes(32))`) guardado en `$_SESSION['_csrf']`. Todos los formularios de la aplicación inyectan un input oculto mediante la función `csrf_field()`. Toda acción que mute el estado del sistema (peticiones POST de creación, actualización o eliminación) invoca obligatoriamente la rutina de control `csrf_verify_or_die()`, ejecutando una comparación segura por tiempo constante mediante `hash_equals()`. Cualquier discrepancia aborta inmediatamente la transacción enviando un código HTTP 403.

- **Mitigación de Cross-Site Scripting (XSS):** Se neutralizan las inyecciones de código malicioso en el navegador forzando a que cualquier salida dinámica de datos pase por la función de escape global `e()` o `e_attr()`. Ambas invocan internamente la rutina `htmlspecialchars` parametrizada con las banderas de seguridad máxima `ENT_QUOTES` | `ENT_SUBSTITUTE` bajo codificación UTF-8, desactivando la interpretación de etiquetas HTML o Javascript peligrosas en el renderizado web.
- **Validación Estricta de Parámetros de Entrada (Input Validation):** El archivo `helpers.php` centraliza la sanitización de inputs mediante funciones fuertemente tipadas que interceptan el tráfico de red: `require_post_int()`, `require_post_decimal()`, `require_post_date()` y `require_post_string()`. Estas funciones utilizan filtros avanzados de PHP (`FILTER_VALIDATE_INT`, expresiones regulares exhaustivas para montos monetarios y validación estricta de objetos de fecha con `DateTimeImmutable`). Si un parámetro es alterado con valores fuera de rango o tipos maliciosos, la ejecución muere en el acto invocando `die_400()` con código HTTP 400.
- **Control de Fuerza Bruta y Rate Limiting:** El proceso de inicio de sesión rastrea las peticiones web mediante la tabla de auditoría `intentos_login`. La función `login_bloqueado()` examina la dirección IP del cliente obtenida por `client_ip()`. Si una IP registra 5 o más fallas consecutivas dentro de una ventana de tiempo de 15 minutos, el sistema bloquea por completo la posibilidad de verificar credenciales. Adicionalmente, ante un error de autenticación válido, se inyecta un retraso de tiempo aleatorio (`usleep(random_int(200000, 600000))`) para mitigar vectores de ataques de temporización (Timing Attacks).
- **Prevención de Insecure Direct Object References (IDOR):** El acceso no autorizado a información de terceros se bloquea aislando las consultas a la base de datos en el contexto estricto de la sesión actual. Todas las sentencias SQL de consulta, actualización o borrado añaden de manera obligatoria la restricción `WHERE usuario_id = ?` o verifican la pertenencia legítima mediante la rutina dedicada `categoria_pertenece_al_usuario()`, impidiendo que un usuario altere identificadores numéricos en la URL para manipular registros financieros de otras cuentas.
- **Cabeceras de Seguridad HTTP:** El servidor Apache inyecta cabeceras de seguridad globales en cada respuesta HTTP para instruir al navegador web: `X-Frame-Options: DENY` (bloquea ataques de Clickjacking), `X-Content-Type-Options: nosniff` (fuerza el respeto a los tipos MIME legítimos), `Referrer-Policy: same-origin` (evita filtraciones de URLs) y remueve de forma activa las cabeceras descriptivas del servidor (`X-Powered-By` y `Server`).

5. Manual de Usuario y Evidencia de Funcionamiento

A continuación se detalla la experiencia de usuario y el comportamiento funcional de los distintos módulos del sistema distribuidos en la capa pública web:

5.1. Control de Acceso (Login)

La interfaz de autenticación presenta un diseño centralizado y limpio. El usuario debe capturar su nombre de usuario único y contraseña. El sistema provee un recuadro informativo con credenciales de demostración seguras (Usuario: demo / Contraseña: Demo2026!) vinculadas

de manera relativa en la base de datos mediante hashing criptográfico avanzado (password_hash con algoritmo por defecto de alta intensidad). Si los datos de sesión son válidos, se regenera el identificador de sesión web para prevenir la fijación de sesiones y se redirige al tablero principal.

5.2. Tablero Principal (Dashboard - Inicio)

El archivo index.php orquesta la visualización consolidada del estado financiero actual del estudiante. Al ingresar, se presentan tarjetas informativas con tres métricas macro destacadas: el Balance General (Diferencia total de ingresos contra gastos), el Total de Ingresos percibidos en el mes corriente y el Total de Gastos ejecutados a la fecha. De forma visual, incorpora un gráfico de distribución circular (Donut Chart) alimentado dinámicamente mediante Chart.js, que desglosa visualmente los porcentajes gastados por categoría. Asimismo, el tablero muestra resúmenes dinámicos del estado de las metas de ahorro con barras de progreso detalladas y un listado cronológico de los próximos pagos recurrentes fijos ordenados por fecha de vencimiento inmediata.

5.3. Gestión Integral de Gastos (Altas, Bajas y Consultas)

La sección de gastos opera bajo un esquema CRUD sumamente granular controlado por tres scripts especializados:

- **Registro (alta.php):** Formulario intuitivo que solicita obligatoriamente el ingreso de un monto numérico decimal positivo, una categoría (obtenida dinámicamente de la base de datos), una descripción textual descriptiva de carácter opcional, la fecha de ejecución (con pre-llenado automatizado al día de hoy) y el método de pago utilizado (efectivo, tarjeta o transferencia bancaria).
- **Historial e Interfaz de Consulta (consulta.php):** Despliega un listado cronológico elegante de todos los cargos asociados a la cuenta. Ofrece controles de filtrado robustos por mes específico, año fiscal y categorías de gasto. Cada registro del listado expone su descripción, fecha exacta, método de pago, un badge de color identificativo y botones directos para disparar acciones individuales de edición o eliminación.
- **Eliminación Avanzada e Individual (eliminar.php):** Provee dos mecanismos altamente flexibles de remoción de datos protegidos. El borrado individual pide confirmación explícita al usuario antes de ejecutar la sentencia. El borrado masivo o por lote (**Bulk Delete**) permite purgar con un solo clic todos los movimientos financieros que coincidan estrictamente con el mes, año y categoría previamente filtrados por el usuario, mostrando un mensaje dinámico informativo con el número exacto de registros eliminados de la sesión.
- **Modificación (editar.php):** Recupera la información íntegra del gasto seleccionado a través de su identificador numérico único, validando previamente su procedencia con la cuenta activa (anti-IDOR). Permite modificar cualquier variable transaccional y guardar de forma segura los cambios impactando en la base de datos de producción de inmediato.

5.4. Administración de Flujos de Ingresos

El módulo ingresos.php unifica en una sola vista la adición y el desglose de flujos entrantes de dinero. Cuenta con controles dedicados para parametrizar la procedencia de los fondos. Una de sus funcionalidades clave es la casilla de verificación para marcar un **ingreso como**

recurrente mensual. Esto permite al software identificar de manera analítica qué flujos de dinero representan fondos fijos garantizados (como becas institucionales recurrentes o mesadas familiares) y cuáles corresponden a entradas esporádicas de efectivo (como trabajos libres de desarrollo web o venta de material usado), impactando directamente los modelos matemáticos predictivos del sistema.

5.5. Control de Límites Financieros (Presupuestos)

El módulo presupuestos.php ayuda a los estudiantes a mantener la disciplina financiera fijando límites máximos de egreso mensuales por categoría. El sistema realiza una consulta agregada cruzando la tabla de presupuestos con la sumatoria en tiempo real de los gastos vinculados a ese mismo mes y año. La interfaz gráfica proyecta una barra de progreso que cambia dinámicamente su estilo y color basándose en el porcentaje de consumo: se renderiza en tonos suaves de gradiente (azul/púrpura) en rangos seguros, cambia a un tono de advertencia ámbar al cruzar el 80% del límite establecido y se transforma automáticamente a un color carmesí de alerta máxima junto a la leyenda presupuesto excedido si los gastos del estudiante superan el 100% de la cuota definida.

5.6. Seguimiento Interactivo de Metas de Ahorro

El archivo metas.php fomenta la planificación económica de mediano plazo (ej. adquisición de computadoras portátiles o cursos de certificación cloud). Las metas calculan su porcentaje de avance y exponen barras visuales de consecución. El sistema implementa una interfaz ágil mediante el script embebido en el footer.php llamado promptMonto(). Al presionar los botones interactivos de **Abonar** o **Retirar**, se despliega una ventana emergente nativa controlada por Javascript que solicita la cantidad al usuario. El script realiza validaciones matemáticas directas en el cliente: bloquea montos negativos, remueve formateos inválidos y restringe el abono para que no supere la cantidad faltante exacta de la meta, ni permite retiros superiores al saldo acumulado actual. Tras ser validado, inyecta el valor de manera segura al backend para actualizar los saldos y recalcular el estado de la meta (marcándola automáticamente como completada si alcanza el 100% del objetivo).

5.7. Análisis Financiero Inteligente (Módulo Serverless)

El panel central de analítica avanzada (analisis.php) representa la cúspide de la integración cloud del sistema, consumiendo los resultados de la computación serverless distribuidos en las siguientes secciones informativas:

- **Indicadores Macroeconómicos (KPIs):** Renderiza tres tarjetas masivas con fuentes tabulares numéricas precisas que exponen el Total Gastado en el periodo evaluado, el promedio exacto de dinero desembolsado por cada movimiento individual registrado y la **Proyección de Cierre de Mes**, un algoritmo matemático predictivo que calcula la tendencia de gasto diaria y estima con cuánta deuda o egreso total terminará el estudiante el mes si mantiene el mismo ritmo de consumo.
- **Comparativa Porcentual Intermensual:** Muestra de forma analítica la variación financiera exacta frente al mes inmediato anterior. Utiliza una lógica visual dinámica con flechas de dirección (↑ para incrementos, ↓ para decrementos) y un código de color inteligente basado en la severidad del cambio (rojo si el gasto escaló drásticamente, verde si se logró contraer el egreso). Adicionalmente, lista un desglose ordenado de las 5

categorías que sufrieron las mayores alteraciones porcentuales para identificar fugas de capital de forma rápida.

- **Evaluación de la Tasa de Ahorro:** Mide la salud financiera del estudiante calculando el porcentaje neto de dinero que se retiene del ingreso total percibido. El backend clasifica el desempeño de forma cualitativa en cuatro niveles estrictos (excelente, buena, baja o negativa), permitiendo alertar al estudiante de forma temprana si se encuentra en un escenario de déficit operativo (gastar más de lo que ingresa).
- **Algoritmo de Proyección contra Quiebra Técnica:** Si el estudiante cuenta con un balance positivo en el mes actual, el software calcula de forma predictiva el promedio diario exacto de sus egresos y evalúa matemáticamente cuántos días calendario de solvencia total le quedan disponibles antes de agotar por completo sus fondos (entrar en quiebra técnica). El sistema cruza este dato contra los días restantes reales del mes en curso, imprimiendo leyendas de tranquilidad si los fondos son suficientes para concluir el periodo, o alertas de contingencia especificando el número exacto de días que el estudiante se quedará corto de dinero.
- **Cumplimiento de la Regla Financiera 50/30/20:** Clasifica dinámicamente los egresos del estudiante bajo un estándar internacional de finanzas personales. Mapea las categorías nativas en tres rubros principales: Necesidades Fijas (comida, transporte, renta, materiales, salud), Deseos Variables (suscripciones, salidas) y Ahorros Acumulados. El panel despliega barras de progreso independientes comparando el porcentaje real consumido por el usuario contra los objetivos ideales estipulados por la regla (50% para necesidades, 30% para deseos y 20% para ahorro), etiquetando cada bloque con un estado de control dinámico (ok, alto o bajo).
- **Detección Estadística de Anomalías de Gasto:** El motor analítico ejecuta un procesamiento estadístico de distribución de datos sobre el conjunto de movimientos del estudiante. Calcula de manera exacta la media aritmética (μ) y la desviación estándar (σ) de los montos económicos. Cualquier registro individual cuyo valor supere el umbral crítico de la media más dos desviaciones estándar ($\text{monto} > \mu + 2\sigma$) es catalogado automáticamente como un **Gasto Inusualmente Alto / Anomalía Estadística**, listándolo de forma prioritaria en un panel de revisión con su descripción y fecha para alertar sobre posibles compras compulsivas o cargos erróneos.
- **Factibilidad de Consecución de Metas:** Cruza el balance de ahorro mensual real del estudiante contra los objetivos de sus metas activas. Si el balance es negativo, le notifica de forma explícita que bajo ese ritmo es matemáticamente imposible ahorrar. Si el balance es saludable, utiliza un objeto de fecha avanzado para proyectar el año y mes exacto en el que completará la meta de ahorro, comparándola contra su fecha objetivo deseada para determinar con precisión milimétrica si la meta se alcanzará A tiempo o se consolidará Tarde.

6. Conclusiones Técnicas

El desarrollo e implementación de StudentWallet demuestra la viabilidad y alta eficiencia de adoptar arquitecturas modernas en la nube para resolver problemas cotidianos con altos estándares de ingeniería de software:

1. **Eficiencia de Costos y Escalabilidad mediante Enfoque Híbrido:** Al delegar los cálculos estadísticos complejos y la predicción algorítmica a AWS Lambda, el servidor

principal web disminuye drásticamente el uso de CPU y memoria. La naturaleza Serverless de Lambda garantiza un esquema de pago por uso exacto y una escalabilidad horizontal infinita y automática, permitiendo procesar miles de transacciones de estudiantes de forma simultánea sin degradar la infraestructura web base.

2. **Seguridad como Pilar Fundamental del Sistema:** La aplicación de principios de *Security by Design* (Seguridad por Diseño) desde la concepción del esquema relacional y el aislamiento de directorios en Apache, hasta las rutinas estrictas de sanitización de inputs y control CSRF en PHP, prueba que es posible desplegar aplicaciones en producción altamente robustas y resilientes ante los vectores de ataque web más comunes e intrusivos del ecosistema informático actual.
3. **Optimización de Recursos Cloud de Almacenamiento:** La integración nativa con Amazon S3 y el uso estratégico de URLs firmadas temporalmente soluciona el problema de la persistencia y la seguridad de los archivos multimedia o documentos en la nube. Esto evita saturar los discos duros locales de las instancias de cómputo EC2 y garantiza un control de acceso centralizado, auditable y seguro que expira de forma automatizada, cumpliendo con las mejores prácticas corporativas de resguardo de información.